

The `Array.prototype.push()` method in JavaScript adds one or more elements to the end of an array and returns the new length of that array. It directly modifies (mutates) the original array instead of creating a new one.

Syntax

`array.push(element1, element2, ..., elementN)`

Key Characteristics

- **Mutates Data:** It alters the original array's content and size.
- **Return Value:** It outputs a number representing the updated length of the array.
- **Multi-Element Support:** You can pass multiple values at the same time, separated by commas

Example

```
const fruits = ['apple', 'banana'];
const newLength = fruits.push('orange');

console.log(fruits);
// Output: ['apple', 'banana', 'orange']

console.log(newLength);
// Output: 3
```

```
const numbers = [1, 2];
numbers.push(3, 4, 5);
console.log(numbers);
// Output: [1, 2, 3, 4, 5]
```

Extra...

Merging Arrays with the Spread Operator (...)

If you push an entire array directly, it nests the array. To append its individual elements instead, use the ES6 spread operator:

```
const primaryColors = ['red', 'blue'];
const secondaryColors = ['green', 'yellow'];

// Spread items into the push method
primaryColors.push(...secondaryColors);

console.log(primaryColors);

// Output: ['red', 'blue', 'green', 'yellow']
```

`loadImage()` and `image()` are two of the most commonly used functions for displaying images.

`loadImage("filename");`

Loads an image from the **data** folder.

Returns a **PImage** object.

`image(img, x, y, width, height);`

Displays an image on the window.

Parameter	Meaning
<code>img</code>	<code>PImage</code> object
<code>x</code>	X-coordinate
<code>y</code>	Y-coordinate
<code>width</code>	Display width (optional)
<code>height</code>	Display height (optional)

```
PImage bird;
void setup() {
  size(700,500);
  bird = loadImage("bird.jpg");
}
void draw() {
  background(220);
  image(bird, 100, 100, 200, 150);
}
```

The `imageMode()` function changes how Processing.js interprets the coordinate parameters inside the `image()` function, altering the anchor point used to position and size your images.

Syntax

`imageMode(MODE);` // Modes: CORNER, CORNERS, CENTER

Visual Structure Modes

- `imageMode(CORNER)`
 - This is the default system mode.
 - Parameters interpretation: `image(img, x, y, width, height)`.
 - `(x, y)` sets the top-left corner of the image.
 - `width` and `height` set the absolute pixel size.
- `imageMode(CORNERS)`
 - Parameters interpretation: `image(img, x1, y1, x2, y2)`.
 - `(x1, y1)` sets the top-left corner location.
 - `(x2, y2)` sets the bottom-right corner location.
 - The image stretches or shrinks to fit exactly between these two points.
- `imageMode(CENTER)`
 - Parameters interpretation: `image(img, x, y, width, height)`.
 - `(x, y)` sets the exact center point of the image.
 - `width` and `height` set the absolute pixel size.
 - Ideal for drawing icons, characters, or rotating images around their middle axis.

Code Example

javascript

```
PImage photo;
void setup() {
  size(400, 400);
  photo = loadImage("avatar.png");
}

void draw() {
  background(220);

  // Example 1: Draw from top-left corner
  imageMode(CORNER);
  image(photo, 10, 10, 100, 100);

  // Example 2: Draw precisely centered on the mouse position
  imageMode(CENTER);
  image(photo, mouseX, mouseY, 80, 80);
}
```

Modifying arrays means changing the data stored inside an array after you have created it

The 3 Ways to Modify an Array

- **Change a value:** Replace an old item with a new one at a specific spot.
- **Add a value:** Grow the array by throwing a new item onto the end.
- **Remove a value:** Shrink the array by deleting an item.

```
var iceCream = ["Vanilla", "Chocolate", "Strawberry"];

// Change "Chocolate" (index 1) to "Mint"
iceCream[1] = "Mint";

// The array is now: ["Vanilla", "Mint", "Strawberry"]
```

Adding a Value (Using `.push()`)

```
var shoppingList = ["Milk", "Eggs"];

// Add "Bread" to the end
shoppingList.push("Bread");

// The array is now: ["Milk", "Eggs", "Bread"]
```

Removing a Value (Using `.pop()`)

```
var cleanUp = ["Paper", "Plastic", "Trash"];

// Remove the last item
cleanUp.pop();

// The array is now: ["Paper", "Plastic"]
```

Here is a quick reference guide on how to modify arrays using the concepts and structure taught on [Khan Academy](#):

1. Adding Elements with `.push()`

The `.push()` method is a built-in command that instructs the computer to add a new value onto the very end of your array, automatically making the array longer. This is famously used in the "Make It Rain" project or balloon popping exercises to track mouse clicks.

```
// Start with an array of x positions  
  
var xPositions = [100, 200, 300];  
  
// Push a new number to the end of the array  
  
xPositions.push(400);  
  
// Now xPositions is [100, 200, 300, 400]
```

2. Common Khan Academy Interactive Usage:

To dynamically draw shapes wherever a user clicks their mouse, you push the mouse coordinates into the array inside the `mouseClicked` or `draw` function:

```
var mouseClicked = function() {  
  
    xPositions.push(mouseX);  
  
    yPositions.push(mouseY);  
  
};
```